

Autonomous Ground Rover

ESP32 + micro-ROS + ROS 2 Kilted + SLAM Toolbox + Nav2

| 2026

1. Goals and Objectives

The primary goal of this project was to design and build a fully autonomous ground rover capable of simultaneously mapping an unknown environment and navigating within it in real time, commonly referred to as Simultaneous Localization and Mapping (SLAM). The project was undertaken as an engineering challenge with the intent of demonstrating a complete autonomous robotics stack on a custom hardware platform.

Specific objectives included:

- Design and implement a differential drive rover using commercially available hobby hardware
- Integrate a 2D LiDAR sensor for real-time environment perception
- Establish a wireless communication bridge between embedded firmware and a ROS 2 host system using micro-ROS over WiFi/UDP
- Deploy SLAM Toolbox for online mapping and localization
- Implement Nav2 with MPPI controller for autonomous path planning and obstacle avoidance
- Calibrate odometry and tune the full navigation stack to achieve reliable autonomous navigation in an indoor environment

2. Introduction

Autonomous mobile robotics presents a unique combination of hardware, firmware, and software engineering challenges. Unlike simulation-based projects, physical robots introduce real-world constraints including sensor noise, mechanical imprecision, wireless communication latency, and timing synchronization across heterogeneous systems. This project was designed to engage with those constraints directly.

The rover was built around an ESP32 microcontroller running custom firmware that interfaces with an RPLidar C1 2D LiDAR sensor and a pair of Feetech STS3215 serial bus servo motors configured in differential drive. The firmware publishes LiDAR scan data and wheel odometry over WiFi using micro-ROS, a framework that bridges embedded systems to the Robot

Operating System (ROS 2). On the host side, a ROS 2 Kilted stack runs SLAM Toolbox for simultaneous mapping and localization, a robot_localization Extended Kalman Filter (EKF) for sensor fusion, and the Nav2 navigation framework with the MPPI (Model Predictive Path Integral) controller for autonomous path planning and execution.

The choice to use an ESP32 rather than a more capable single-board computer such as a Raspberry Pi was deliberate. The ESP32 introduced significant engineering constraints — limited compute, no operating system, WiFi-only communication, and clock drift between the microcontroller and host — that required creative solutions and deep understanding of the full system. These constraints drove much of the project's technical depth.

3. Method

3.1 Hardware Architecture

The rover platform consists of two independently driven wheels mounted on a rigid chassis, with the following components:

- ESP32 microcontroller — central embedded compute, WiFi communication
- RPLidar C1 — 2D LiDAR sensor, 360° scan, up to 12m range, connected via UART at 460800 baud
- Feetech STS3215 servo motors — serial bus servos in wheel mode, providing both actuation and absolute position feedback for odometry
- 5V/3A power bank — powers ESP32 and LiDAR
- 2x 18650 Li-Ion cells in series (7.4V) — powers servo bus

The LiDAR and servo bus share the ESP32's hardware serial peripherals on dedicated UART pins. The servo bus uses a half-duplex serial protocol at 1 Mbaud. Both power rails share a common ground with the ESP32 to ensure reference consistency.

3.2 Firmware Architecture

The ESP32 firmware was written in C++ using the Arduino framework with the micro-ROS Arduino library. The firmware performs four primary functions in a single cooperative loop:

- LiDAR data acquisition — the LDS_RPLIDAR_C1 library processes incoming serial data and fires a callback per scan point, accumulating a 360-element range array which is published as a sensor_msgs/LaserScan message at 5 Hz
- Odometry calculation — absolute servo position is read at 10 Hz, tick differences are computed with wraparound handling, and differential drive kinematics produce x, y, and heading estimates published as nav_msgs/Odometry
- Velocity command handling — geometry_msgs/Twist messages on /cmd_vel are received via micro-ROS executor and converted to per-wheel speed commands using differential drive inverse kinematics, sent to the servos as signed 16-bit speed values via the STS protocol
- Clock synchronization — rmw_uros_sync_session() is called every 500ms to align the ESP32 clock with the ROS host, mitigating timestamp drift that would otherwise cause SLAM Toolbox to drop scans

A key firmware discovery during development was that the Feetech STS library's WriteSpe() function expects a signed 16-bit integer (s16) for direction encoding, not manual bit manipulation. An incorrect implementation using bitwise sign encoding was corrupting motor speed commands throughout early testing, causing asymmetric wheel speeds and degraded navigation performance.

3.3 ROS 2 Stack

The host-side ROS 2 stack runs on a Samsung 950QDB laptop (Intel i7-1165G7, 4 cores / 8 threads) and consists of the following nodes:

- micro-ROS agent — UDP/IPv4 bridge between the ESP32 and ROS 2 DDS middleware
- static_transform_publisher — publishes the fixed base_link → laser_frame transform based on physical LiDAR mounting position (7.5cm forward, 14cm height)
- robot_localization EKF — fuses wheel odometry into a filtered pose estimate, publishing the odom → base_link transform at ~13.5 Hz
- SLAM Toolbox (async_slam_toolbox_node) — performs online scan matching to build a 2D occupancy grid map and publishes the map → odom transform
- Nav2 stack — controller_server (MPPI), planner_server (NavFn), behavior_server, bt_navigator, velocity_smoother, collision_monitor, and supporting nodes

The cmd_vel pipeline routes commands through the following chain: controller_server publishes to cmd_vel_nav, which is consumed by velocity_smoother, which outputs to cmd_vel_smoothed, which is consumed by collision_monitor, which outputs to the final cmd_vel topic subscribed to by the ESP32.

3.4 Odometry Calibration

Wheel odometry accuracy is critical for SLAM and navigation. Two parameters govern odometry quality: WHEEL_RADIUS (affects linear scale) and TRACK_WIDTH (affects angular accuracy). Both are defined as constants in the ESP32 firmware.

TRACK_WIDTH was calibrated using a multi-rotation spin test. The rover was commanded to spin 1080° (3 full rotations) in place using a timed velocity script. The actual angle turned was measured visually, and TRACK_WIDTH was scaled proportionally:

$$new_track = old_track \times (actual_angle / commanded_angle)$$

The effective track width (0.1875m) is larger than the physical wheel-to-wheel distance due to lateral wheel slip during in-place rotation, which is expected behavior for differential drive robots on hard floors. The multi-rotation test was used because it amortizes fixed start/stop lag errors, producing a more accurate per-revolution correction factor.

3.5 Nav2 Configuration

The Nav2 stack was configured for a small, slow differential drive robot operating in an indoor environment. Key configuration decisions included:

- MPPI controller with DiffDrive motion model, batch_size=200, time_steps=10, vx_max=0.172 m/s (measured physical maximum)
- transform_tolerance=1.0s across all Nav2 components to accommodate ESP32/host clock drift
- enable_stamped_cmd_vel=false on all velocity nodes (ROS 2 Kilted defaults to TwistStamped; ESP32 subscribes to plain Twist)
- Goal tolerances: xy_goal_tolerance=0.3m, yaw_goal_tolerance=0.5 rad, appropriate for the odometry accuracy achievable with the hardware
- SLAM Toolbox configured with do_loop_closing=false during navigation to prevent sudden map frame shifts that destabilize the MPPI controller

4. Results

The rover ultimately achieved stable real-time SLAM, reliable obstacle avoidance, and consistent response to manual velocity commands. The system maintained stable SLAM operation with consistent 10 Hz publication on both the /scan and /odom topics, each measured with an observed standard deviation of approximately 0.05 Hz. SLAM Toolbox produced coherent occupancy grid maps during teleoperation, while Nav2 successfully generated collision-free trajectories and maintained consistent obstacle avoidance behavior in indoor environments. Odometry quality improved substantially after empirical track width calibration and resolution of motor control bugs, resulting in reduced angular drift and improved localization stability.

However, fully autonomous goal navigation remained inconsistent. During autonomous operation, the interaction between online SLAM updates, ESP32 clock synchronization drift, TF timing tolerances, and Nav2 controller timing introduced intermittent instability, particularly during in-place rotations and recovery behaviors. Much of the development process therefore focused on diagnosing timing, transform, odometry, and message-type issues across the ROS 2 navigation stack. The following sections summarize the major technical problems encountered during development, along with the solutions implemented and the engineering reasoning behind each change.

Rover Project — Issue & Resolution Log

1. Odometry & Wheel Calibration

Issue	Solution / Change	Justification
Angular drift during navigation	Iterative spin-in-place calibration: adjusted TRACK_WIDTH in ESP32 sketch from 0.18 to 0.1875m	TRACK_WIDTH is the divisor in $\text{delta_theta} = (\text{d_left} - \text{d_right}) / \text{TRACK_WIDTH}$. Too large = undershoot, too small = overshoot. Wheel slip during in-place rotation means effective

Issue	Solution / Change	Justification
		track is always larger than physical measurement.
URDF wheel_track changes had no effect on odometry	Identified that odometry is computed on the ESP32, not by ROS. Changed TRACK_WIDTH constant in Arduino sketch instead.	The URDF joint Y-offset only affects TF/visualization. The actual odometry math runs in calculate_odometry() on the ESP32 and uses its own TRACK_WIDTH define.
Persistent undershoot despite track width changes	Switched from 1-rotation to 3-rotation test; scaled correction proportionally	Single rotation test has high start/stop lag error relative to signal. 3 rotations amortizes fixed lag error, making the per-revolution error more accurate. Error scaled linearly confirming it was a track width issue, not lag.
Wheel axis defined as <0 0 1> in URDF	No change needed — rpy='-1.5708 0 0' on the joint rotates the frame so local Z aligns with world Y	The -90deg X rotation means the wheel's local Z axis points along world Y (the axle direction), so axis 0 0 1 in local frame = rotation around world Y. Functionally correct despite appearing wrong.
Lidar offset in URDF was xyz='0 0.075 0.14' (sideways)	Corrected to xyz='0.075 0 0.14' (forward)	X=forward, Y=left/right in ROS convention. A sideways offset causes SLAM to compute wrong scan-to-base transforms, misaligning the costmap footprint relative to actual obstacles.

2. cmd_vel & Motor Control

Issue	Solution / Change	Justification
Python calibration script did nothing; no messages on /cmd_vel	Replaced create_rate()+rate.sleep() with time.sleep(0.1) loop; added 0.5s startup delay	create_rate().sleep() requires the node to be spinning in a separate thread to function. time.sleep() works without a spinner. Startup delay gives the publisher time to establish connection before first message.

Issue	Solution / Change	Justification
Type conflict on /cmd_vel: both Twist and TwistStamped present	Added enable_stamped_cmd_vel: false to behavior_server params	ROS 2 Killed defaults behavior_server to TwistStamped. The ESP32 subscribes to plain Twist. Having both types on one topic causes subscribers to reject messages of the wrong type silently.
No direct distance/angle command interface	Used timed velocity commands: distance = speed x duration, angle = (radians / speed) seconds	cmd_vel is a pure velocity interface. Distance/angle control requires either timed commands (open loop) or Nav2 actions (closed loop via odometry). Open loop used for calibration to avoid circular dependency on the odometry being calibrated.

3. Nav2 Configuration

Issue	Solution / Change	Justification
TF extrapolation error: map->odom transform 592ms stale	Increased transform_tolerance to 1.0s across all Nav2 components; increased ESP32 clock sync frequency to every 1-2s	ESP32 and host clocks drift causing timestamps to diverge. transform_tolerance tells Nav2 to accept transforms up to Ns old. rmw_uros_sync_session() corrects the ESP32 clock but must run frequently enough to prevent drift exceeding the tolerance.
transform_tolerance missing from local_costmap and global_costmap	Added transform_tolerance: 1.0 to both costmap configs	Costmaps do their own TF lookups to transform laser scans into the costmap frame. Without their own tolerance setting they use a very short default, causing them to reject scans during clock drift events.
min_y_velocity_threshold: 0.5 in controller_server	Changed to 0.001	A differential drive robot has zero lateral (Y) velocity by definition. Setting this to 0.5 told the controller to ignore all lateral velocity feedback under

Issue	Solution / Change	Justification
		0.5 m/s, which on a diff drive means ignoring all velocity feedback entirely. Likely caused failed-to-make-progress errors.
Controller loop running at 2 Hz instead of 10 Hz	Fixed by correcting enable_stamped_cmd_vel and type conflict issues	The Twist/TwistStamped type conflict and misconfigured params were causing the controller to stall. At 2 Hz the rover overshoots turns before correction, causing oscillation near goals and map degradation during autonomous nav.
MPPI batch_size: 300 contributing to slow control loop	Reduced to 200; time_steps reduced from 15 to 10	MPPI computation scales with batch_size x time_steps. At the rover's max speed of 0.3 m/s, 10 steps x 0.1s dt = 1.0s lookahead is sufficient. Reducing batch from 300 to 200 cuts computation by 33% with minimal trajectory quality loss at slow speeds.
Navigation aborting with error 102: unable to transform goal into costmap frame	Increased transform_tolerance; increased clock sync frequency on ESP32	Error 102 is thrown when the controller cannot transform the goal pose into the costmap frame due to a stale TF. Root cause is clock drift between ESP32 and host. Both the tolerance increase and faster sync are needed: tolerance handles transient drift, sync prevents accumulated drift.
Navigation aborting with error 105: failed to make progress	Loosened progress_checker: required_movement_radius 0.3, movement_time_allowance 10.0s; loosened goal tolerances to xy=0.15m, yaw=0.2rad	Error 105 fires when the rover does not move required_movement_radius within movement_time_allowance. At 2 Hz controller frequency with type conflict bugs active, the rover was not receiving

Issue	Solution / Change	Justification
		commands consistently enough to make reliable progress.
behavior_server spin max_duration: 5.0 arbitrarily killing recovery behaviors	Removed max_duration parameter	A 5-second cap on recovery spins would abort the behavior before completion on any spin > ~2.8 radians at min_rotational_vel of 0.4 rad/s. Recovery behaviors should run to completion.

4. SLAM Toolbox

Issue	Solution / Change	Justification
Map degrades during autonomous navigation, especially during in-place rotation	Increased minimum_travel_heading to 0.3 rad; added minimum_time_interval: 0.2s; tightened loop_match_minimum_response_fine to 0.45	During Nav2-commanded spins, scans arrive faster than the matcher can process them. minimum_travel_heading reduces processing rate during rotation. minimum_time_interval enforces a floor on scan processing frequency. Higher loop match threshold prevents bad loop closures from corrupting the map.
transform_timeout: 0.5 insufficient for clock drift	Increased to 1.0	SLAM Toolbox drops scans when it cannot look up the base_link->odom transform within transform_timeout. With up to 592ms of observed clock drift, 0.5s timeout was causing frequent scan drops exactly when the rover was moving and generating the most useful mapping data.
map_update_interval: 5.0s too slow for	Reduced to 3.0s	During autonomous nav the global costmap needs reasonably fresh map data

Issue	Solution / Change	Justification
autonomous navigation		to plan around newly observed obstacles. 5s intervals meant the planner could be working from a map that was missing recently seen walls or obstacles.
YAML integer vs double type error on transform_timeout	Ensured all float params have explicit decimal points (e.g. 1.0 not 1)	ROS 2 parameter typing is strict and does not coerce integers to doubles. YAML interprets any number without a decimal as an integer. Setting a double parameter to an integer value throws an invalid type error and prevents the node from starting.

5. Conclusions and Future Work

5.1 Conclusions

This project successfully demonstrated a complete autonomous mobile robotics stack on a resource-constrained embedded platform. The rover achieved reliable 2D mapping via SLAM Toolbox, consistent obstacle avoidance, and attempted autonomous navigation to goal poses. Several key engineering insights emerged from the development process:

- Clock synchronization between an ESP32 and a ROS 2 host is a persistent and non-trivial problem. Timestamp drift directly impacts SLAM scan matching, TF transform validity, and Nav2 controller frequency. The transform_tolerance parameter must be set aggressively across every Nav2 component to accommodate this drift.
- The cmd_vel pipeline in Nav2 is a non-obvious multi-hop chain. A remapping inconsistency between the velocity smoother and collision monitor caused commands to be silently dropped for extended periods, producing navigation behavior that appeared random. Verifying every topic in the chain with ros2 topic hz is essential.
- Odometry on servo-based differential drive is viable but requires careful calibration. The effective kinematic track width differs from the physical measurement due to wheel slip and must be determined empirically. The bitwise sign encoding bug in the motor control layer demonstrates that low-level firmware errors can produce high-level navigation failures that are difficult to trace.
- SLAM and Nav2 have a fundamental tension in online mapping mode. SLAM Toolbox continuously updates the map→odom transform as it refines the map, which can shift

the reference frame mid-navigation. Disabling loop closure during navigation reduces map consistency but significantly improves navigation stability.

- The EKF from `robot_localization` provides a valuable filtering layer between raw odometry and the Nav2 stack, smoothing noisy wheel encoder data and providing a stable `odom`→`base_link` transform even during periods of ESP32 communication latency.

5.2 Future Work

Several improvements have been identified for future development:

Migration to Raspberry Pi 4: The ESP32's WiFi/UDP communication layer introduced the majority of the project's most difficult integration problems. A Raspberry Pi 4 running ROS 2 natively eliminates clock drift, reduces latency, and provides sufficient compute headroom for MPPI, SLAM, and Nav2 to run simultaneously without competing for CPU.

IMU Integration: Adding an IMU to the EKF sensor fusion would significantly improve odometry during rapid rotation, where wheel slip causes the largest accumulated errors. The `robot_localization` EKF supports IMU fusion natively with minimal configuration changes.

Two-phase Navigation: For production use, separating map building from navigation (manual mapping followed by localization-only Nav2 operation) would produce more stable and predictable navigation behavior. Online SLAM during navigation is technically impressive but introduces map consistency challenges that are difficult to fully mitigate through parameter tuning alone.

Encoder-based Odometry: The STS3215 servos provide absolute position feedback, which requires wraparound handling and is susceptible to blocking I/O latency. Direct quadrature encoder feedback from dedicated motor encoders would provide higher resolution, lower latency odometry with no wraparound complexity.

Nav2 Parameter Autotuning: The large number of interdependent Nav2 parameters (MPPI critic weights, costmap tolerances, controller frequency, goal tolerances) makes manual tuning time-consuming. A systematic Bayesian optimization approach or navigation benchmarking suite would accelerate parameter convergence.

6. Code and Setup

6.1 Software Setup and Installation

Due to the strict nature of ROS 2 workspace structure, code excerpts in this report are intended for reference only. The full project can be found at <https://github.com/AnkhC1/rover-project>. The

branch 'main' contains mapping functionality only; 'nav2-integration' reflects the complete state of the project as described in this report.

The repository structure requires a note for anyone attempting to reproduce this work. The `~/rover-project/` directory serves as the Git root for version control purposes, consolidating all project files in one place. However, the `rover_bringup` ROS 2 package cannot be built from within this directory. The actual ROS 2 workspace lives at `~/rover_ws/`, which was created independently and contains the standard ROS 2 workspace structure (`src/`, `build/`, `install/`, `log/`). The `rover_bringup` package inside `~/rover_ws/src/` is a symlink pointing into `~/rover-project/`, allowing edits to be version-controlled through Git while the workspace remains functional. To build the package, `colcon build --symlink-install` must be run from `~/rover_ws/`, not from `~/rover-project/`.

[Installation instructions for Ubuntu 24.04 LTS](#)

[Installation instructions for ROS2 Kilted on Ubuntu](#)

[Installation instructions for building micro ros agent from source](#) (Note: The specific installation path will differ depending on the chosen microcontroller, IDE, and ROS2 version)

On Ubuntu, the `slam_toolbox` and `Nav2` packages can be installed simply by:

```
sudo apt update
sudo apt install ros-kilted-slam-toolbox
sudo apt install ros-kilted-navigation2 ros-kilted-nav2-bringup
```

(The `<ros-kilted>` version tag may be replaced by whichever ROS2 version is chosen.)

Development Environment

- Operating System: Ubuntu Desktop 24.04 LTS
- ROS Distribution: ROS 2 Kilted Kaiju
- Microcontroller: ESP32-S v1.1 (NodeMCU form factor)
- Arduino IDE Version: 2.3.4

Arduino Libraries

- LiDAR driver library: <https://github.com/kaiaai/LDS>
- micro-ROS Arduino library: https://github.com/micro-ROS/micro_ros_arduino
- Servo communication library: SCServo library provided by Waveshare

Serial Communication Configuration

Device	Interface	Baud Rate
RPLidar C1	UART	460800 baud
STS3215 Servo Bus	Half-duplex UART	1000000 baud

WiFi and Network Topology

The rover communicates with the ROS 2 host laptop using micro-ROS over WiFi/UDP. The ESP32 and host computer must be connected to the same wireless network, or the ESP32 may be connected to a hotspot created by the host computer. The micro-ROS agent runs on the host laptop and bridges UDP packets from the ESP32 into the ROS 2 DDS middleware.

ESP32 → WiFi/UDP → micro-ROS Agent → ROS 2 Nodes

ROS 2 Workspace Build Instructions

The ROS 2 workspace was built using `colcon` with symbolic linking enabled:

```
cd ~/rover_ws
```

```
source /opt/ros/kilted/setup.bash
colcon build --symlink-install
source install/setup.bash
```

micro-ROS Agent Launch

```
ros2 run micro_ros_agent micro_ros_agent udp4 --port 8888
```

Expected ROS 2 Topics

Topic	Message Type	Description
/scan	sensor_msgs/LaserScan	LiDAR scan data
/odom	nav_msgs/Odometry	Wheel odometry
/cmd_vel	geometry_msgs/Twist	Final velocity command to ESP32
/tf	tf2_msgs/TFMessage	Dynamic transforms
/tf_static	tf2_msgs/TFMessage	Static transforms
/map	nav_msgs/OccupancyGrid	SLAM-generated map
/cmd_vel_nav	geometry_msgs/Twist	Raw Nav2 controller output
/cmd_vel_smoothed	geometry_msgs/Twist	Velocity-smoothed commands

Repository Structure

The full project repository is available at:

<https://github.com/AnkhC1/rover-project>

The 'main' branch contains mapping functionality only, while the 'nav2-integration' branch reflects the complete autonomous navigation configuration described in this report.

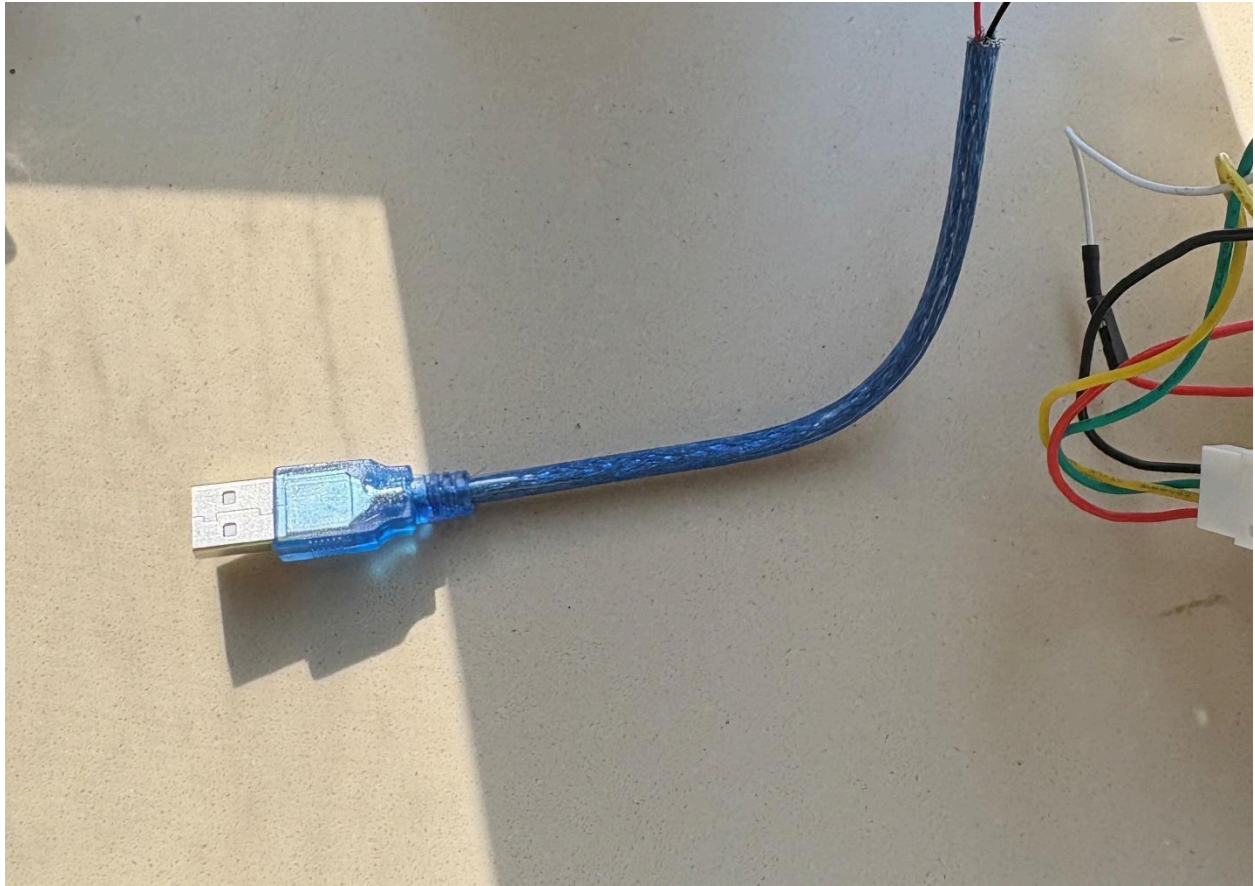
6.2 Hardware Setup

Chassis:

<https://cad.onshape.com/documents/83125d1df7da4e076211cfb5/w/6ad0806186d95563e5e967b6/e/4eaaa43f1ce4cf23ad352b7c>

The hardware setup for this project was extremely simple and therefore easily modified. It was a simple differential drive rover (2 wheels and a caster) with a 2-layer geometry to prevent hardware from protruding into the FOV of the LiDAR, which was mounted on the top layer.

The ESP32 and LiDAR were simultaneously powered by a SIXTHGU USB power bank (<https://www.amazon.com/SIXTHGU-Portable-Charger-Charging-Flashlight/dp/B0C7PHKKNK>). Note that this setup requires a USB-to-Dupont breakout cable from the USB power bank. It is possible to repurpose a spare USB-to-Serial connector from an Arduino for this (pictured below), but it is not recommended.



The STS3215 motors received power from a 7.4V 18650 battery pack directly through the power header on the Waveshare Bus Servo Adapter v1.1 (A). Ensure that this is a dedicated power delivery circuit to prevent inductive spikes on the ESP32 circuit.

GPIO Pins 16 (RX) and 17 (TX) were assigned using hardware serial on the ESP32, connected to TX and RX on the LiDAR, respectively. The same was done for pins 18 (TX) and 19 (RX) on the ESP32 to the bus servo adapter (ensuring TX -> RX both ways). The bus servo adapter, LiDAR, and both power supplies share common ground on the ESP32 to prevent floating ground errors.

6.3 Video

<https://youtube.com/shorts/VPZTrOBSdS8?feature=share>